StreamWeave: Reconciling Off-Policy Expert Supervision with Fully Asynchronous Policy Learning

Anonymous submission

Abstract

Reinforcement learning with verifiable rewards (RLVR) generates multiple rollouts per prompt. But on hard problems, rollouts tend to be long and expensive yet often all fail, concentrating two problems in the same region: high generation cost and the absence of reward contrast within the group. Fully asynchronous RL addresses the cost problem by decoupling rollout generation from model training; expert trajectories address the signal problem by supplying verified solutions for prompts that the policy cannot solve. We study a group-conditioned rule that retains policy rollouts when the complete-group success rate exceeds a threshold and otherwise selects a verified expert trajectory. Treating group completion as the execution boundary would reintroduce the synchronization barrier that asynchronous execution was designed to remove. StreamWeave separates the lifetimes of group waiting and the information needed to preserve the selected learning role: the wait ends once the source is fixed, while that information persists only until the corresponding training input is built. Regardless of source, the resulting inputs enter one update under the same policy objective; an incomplete group delays only its own source decision, while other rollouts are generated and training on ready inputs continues. On mathematical reasoning benchmarks, StreamWeave scores 38.5 on average, comparable to 37.7 for a synchronous reference that applies the same source rule. Under the same GPU budget, it reduces the processing time for 128 prompt groups from 46 to 28 seconds and improves throughput by $1.64\times$. These results show that a group's source decision can remain intact without making group completion a pipeline-wide synchronization barrier.

1 Introduction

Reinforcement learning with verifiable rewards (RLVR) improves the reasoning ability of large language models by reinforcing automatically verifiable successes. Group-based RLVR generates multiple rollouts per prompt and compares their verified outcomes to form a relative learning signal; long reasoning trajectories make this generation a major component of training wall-clock time. On problems that the policy cannot yet solve, these rollouts are often long yet all fail, so the group expends substantial generation cost without producing reward contrast. In our training trace, all-failure groups account for 22.4% of all groups but 26.8% of generated response tokens. Generation cost and missing relative signal therefore concentrate in the same region. Fully asynchronous RL mitigates the former by allowing rollout generation and model training to advance concurrently, while expert trajectories address the latter by supplying verified solutions. We study conditional expert supervision that retains policy rollouts when the complete-group success rate exceeds a threshold and otherwise selects an expert trajectory. This creates a composition problem: the source decision is defined only on a complete group, whereas a fully asynchronous runtime advances attempts independently. Treating group completion as the execution boundary would place the slowest rollout back on the pipeline critical path.

The key observation is that group waiting and the information needed to preserve the selected learning role have different lifetimes: waiting ends once the source is fixed, whereas that information persists until the corresponding training input is built. The source decision determines more than which data are used for training; it determines the form of their update contribution: a group-relative signal with policy-lag correction for policy rollouts, or a weighted supervision signal for an expert trajectory. StreamWeave preserves both the decision and the learning role it selects without re-introducing a synchronization barrier. It executes each attempt independently and reconstructs a complete group when its source can be decided. Once the source is fixed, a source-fixed record carries the selected data and only the context needed to construct its training input through the queue. After the queue, the record is converted into the training input required by the fixed source: a group-relative advantage and per-rollout importance weights for policy data, or a supervision signal for expert data. Inputs from either source are consumed by one policy objective. By localizing composition-specific processing at the queue boundaries, StreamWeave leaves both engines in their native roles: the generator produces attempts and the trainer optimizes prepared inputs. Thus, only that group's source decision is delayed; generation and training on ready inputs continue.

On mathematical reasoning benchmarks, StreamWeave scores 38.5 on average, comparable to 37.7 for HPT, the synchronous quality reference using the same source rule.

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

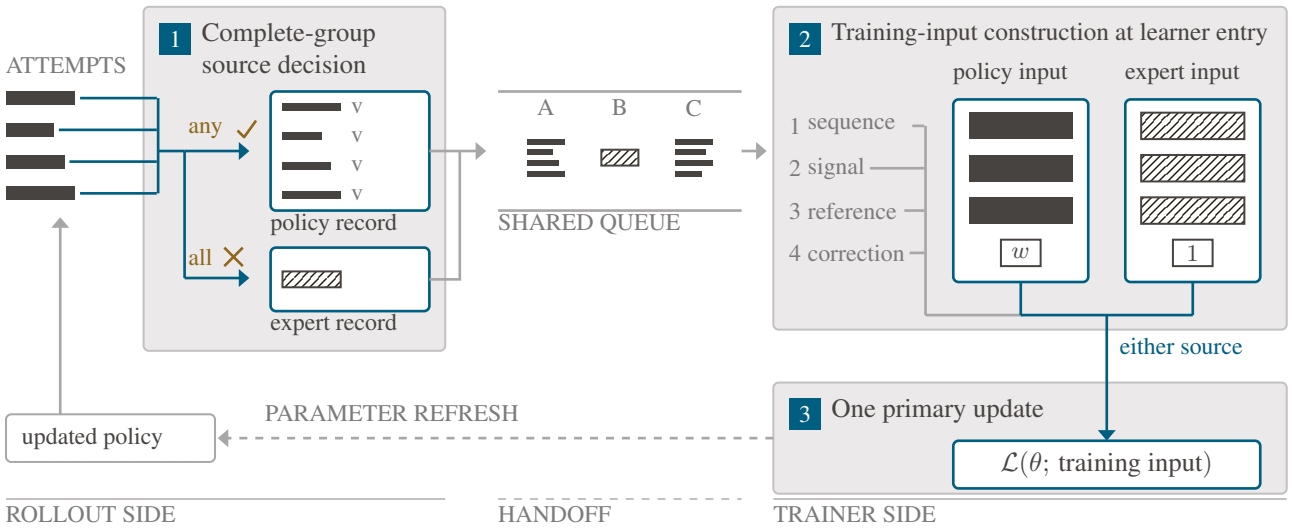

Figure 1: StreamWeave's two source-specific transformations under the 0/8 rule: source decision before the shared queue and training-input construction at learner entry. Both inputs then enter one primary update.

The demand for expert trajectories persists late into training: expert routing remains substantial, and StreamWeave and an expert-off control, initially similar, later separate in StreamWeave's favor in both reasoning performance and held-out all-failure behavior. Under the longer generation workload observed with expert supervision, StreamWeave reduces the processing time for 128 prompt groups from 46 to 28 seconds relative to synchronous execution and improves throughput by $1.64\times$ under the same GPU budget.

StreamWeave allows a complete-group source decision to shape the learning update without setting the pace of the pipeline. **Preserving distinct learning roles in one policy-objective path under full asynchrony.** We carry the information needed to realize the policy or expert learning role selected by a complete group across the asynchronous handoff and encode that role in the training input at learner entry. Section 3.1 derives the intended group-relative policy and weighted supervised contributions under one policy objective and a source-independent normalization, after which source-specific learner control ends and both inputs follow the same learner and optimizer path.

Complete-group decisions without a pipeline-wide barrier. We confine complete-group waiting to the source decision while carrying the context needed for subsequent training-input construction. Resolving the source before the queue and constructing its input after the queue keep the generator and trainer in their native roles; Section 3.2 and Algorithm 1 specify the design.

From persistent expert demand to same-budget execution payoff. We trace a single empirical chain: complete-group source decisions arise in the most generation-intensive region, remain learning-relevant as training proceeds, and make the accompanying long-tail workload a sustained execution condition rather than a transient corner case. Section 4 tests each link with controlled learning comparisons and same-budget system measurements, showing competitive reasoning quality and $1.64\times$ prompt-group throughput.

2 Related Work

StreamWeave addresses a composition problem at the intersection of systems that decouple rollout generation from model training and methods that incorporate expert trajectories into policy learning.

Fully asynchronous policy learning. Asynchronous RLHF (Noukhovitch et al. 2025) decouples generation from learning and studies the trade-off between execution efficiency and policy lag when training on samples from earlier policies. AReaL (Fu et al. 2025) fully disaggregates rollout and training workers and combines workload balancing with staleness-aware optimization in an end-to-end asynchronous system. Laminar (Sheng et al. 2026) independently executes trajectories in GRPO to absorb long-tail variation in generation latency. These systems decouple the production and consumption of policy-generated rollouts while controlling policy lag; none uses a complete-group outcome to select both the training source and its learning role.

Learning from policy rollouts and expert trajectories. Prior work in general RL has combined demonstrations with online learning (Hester et al. 2018; Ball et al. 2023). In LLM reasoning post-training, LUFFY (Yan et al. 2025) combines off-policy reasoning traces with policy rollouts through mixed-policy learning, while CHORD (Zhang et al. 2026) dynamically weights expert supervision alongside on-policy exploration. ReLIFT (Ma et al. 2026) interleaves targeted expert fine-tuning with RL, and SRFT (Fu et al. 2026) simultaneously trains on demonstrations and self-exploration rollouts. HPT (Lv et al. 2025) provides the complete-group source rule and unified learning view instantiated here; these methods report synchronous pipelines and do not define the fully asynchronous handoff addressed by StreamWeave.

Group-conditioned expert supervision under fully asynchronous execution. Fully asynchronous systems decouple when policy rollouts are produced and consumed, whereas expert-guided methods determine how policy and expert data enter learning and contribute to the objective. Under the source rule studied here, both capabilities must coexist on the same hard prompts. All-failure groups are routed to expert trajectories, while their long and uneven rollouts create disproportionate completion pressure. A complete-group outcome therefore couples source selection to execution. Yet neither line specifies where complete-group waiting can end or how long the resolved source and group context must persist. StreamWeave addresses this gap by preserving the selected learning role without making complete-group waiting a pipeline-wide synchronization barrier.

3 StreamWeave

A complete-group source decision imposes two dependencies with different lifetimes: complete-group waiting is required only until the source is fixed, whereas the information needed to preserve the selected learning role must persist until the corresponding training input is constructed. StreamWeave ends each dependency at its required endpoint, allowing execution outside these boundaries to follow the existing fully asynchronous flow (Figure 1).

3.1 Distinct Learning Roles under One Policy Objective

A complete-group decision determines both which data enter training and how they contribute to the update. The two sources differ in the learning role assigned to each sequence, the reference that defines its policy ratio, and whether policy-lag correction applies. StreamWeave encodes every source-specific condition required by the selected role in the training input itself. A source-independent token-level policy objective then exactly recovers group-relative and weighted supervised contributions from the respective inputs.

This section constructs the two inputs, derives these endpoints under the clipped-PPO realization used here, and shows that the primary policy update no longer branches on source once the inputs are formed.

We adopt the complete-group source rule of Hybrid Post-Training (HPT; Lv et al. 2025). Section 4.1 specifies the group size, source threshold, and expert-trajectory coverage.

Source selection from a complete group. For prompt x, let $G_x = \{\tau_{x,1}, \ldots, \tau_{x,n}\}$ denote n policy-generated rollouts. Each rollout has verifier score $R(\tau_{x,i})$, and the success rate of the complete group is

$$\begin{aligned} P_x &= \frac{1}{n}\sum_{i=1}^{n} \mathbf{1}[R(\tau_{x,i}) > \delta]\,, \\ z_x &= S_\gamma(G_x) = \begin{cases} \text{expert}, & P_x \le \gamma, \\ \text{policy}, & P_x > \gamma. \end{cases} \end{aligned} \tag{1}$$

Training uses the generated group G_x when $z_x = \text{policy}$ and the expert trajectory $\tau_x^\star$ paired with the same prompt when $z_x = \text{expert}$. Because the rule is evaluated on G_x as a complete group, StreamWeave resolves the source after all n verifier scores have arrived.

Constructing source-specific training inputs. For token t in selected sequence r, let $\ell_{\theta,r,t}$ be its log-probability under the current policy, $A_{r,t}$ its per-token signal, $\rho_{r,t}$ its learner-entry policy ratio, and $w_{r,t}$ a source-specific multiplicative weight.

For the policy source, StreamWeave retains all n rollouts and compares their verifier scores to compute the GRPO advantage (Shao et al. 2024) $A^{\text{grp}}_{r,t}$. Let $\ell^{\text{gen}}_{r,t}$ be the log-probability under the policy that generated a rollout and $\ell^{\text{entry}}_{r,t}$ its log-probability under the policy snapshot taken when the learner begins the update. We use the entry policy as the ratio reference and correct generation-to-entry lag with a truncated token-level importance ratio:

$$\begin{aligned} A_{r,t} &= A^{\text{grp}}_{r,t}, \\ \rho_{r,t} &= \exp\!\left(\ell_{\theta,r,t} - \ell^{\text{entry}}_{r,t}\right), \\ w_{r,t} &= w^{\text{lag}}_{r,t} = \min\!\left(C_w,\, \exp\!\left(\ell^{\text{entry}}_{r,t} - \ell^{\text{gen}}_{r,t}\right)\right). \end{aligned} \tag{2}$$

Here, C_w is the truncation cap specified in Appendix A.2.

An expert trajectory is an externally supplied target sequence rather than past experience generated by the policy. Its role is therefore to increase the likelihood of the target directly, without a group-relative advantage or rollout-to-entry correction. Using the supervision strength β_r as its signal and a stop-gradient copy of the current-policy log-probability as its reference makes the forward ratio equal to one:

$$\begin{aligned} A_{r,t} &= \beta_r, & w_{r,t} &= 1, \\ \rho_{r,t} &= \exp(\ell_{\theta,r,t} - \text{sg}[\ell_{\theta,r,t}]) = 1. \end{aligned} \tag{3}$$

One policy objective. Inputs from both sources enter the same token-level policy objective Φ. For policy rollouts, our experiments use Decoupled PPO (Fu et al. 2025): $w_{r,t}$ corrects rollout-to-entry policy lag, and Φ applies clipped PPO (Schulman et al. 2017) to the learner update. The batch objective is

$$\mathcal{L}(B) = \frac{1}{|B| L_{\max}} \sum_{r \in B} \sum_{t} m_{r,t} \Phi(\rho_{r,t}; A_{r,t}, w_{r,t})\,, \tag{4}$$

where $m_{r,t}$ is the response mask and $L_{\max}$ is a fixed maximum-response-length divisor shared by both sources. Appendix A.3 specifies its realized value and rationale.

For the clipped-PPO realization used here, substituting the policy input (Eq. 2) into Eq. 4 recovers the group-relative PPO contribution, anchored at the learner-entry policy and weighted by the rollout-to-entry correction $w^{\text{lag}}_{r,t}$.

For the expert input, Eq. 3 gives $\rho_{r,t} = 1$ in the forward pass and $w_{r,t} = 1$. Because the denominator is detached,

$$\nabla_\theta \rho_{r,t} = \rho_{r,t}\, \nabla_\theta(\ell_{\theta,r,t} - \text{sg}[\ell_{\theta,r,t}]) = \nabla_\theta \ell_{\theta,r,t}.$$

Since $\beta_r > 0$ and $\rho_{r,t} = 1$ lies inside the clipping interval, the clipped and unclipped branches coincide locally. Thus, for each active response token,

$$\nabla_\theta \Phi_{\text{PPO}}(\rho_{r,t}; \beta_r, 1) = -\beta_r\, \nabla_\theta \ell_{\theta,r,t}.$$

Algorithm 1 StreamWeave control flow; bold boundary labels match Figure 1.

Require: Prompt pool $\mathcal{X}$, group size n, verifier R, source rule S_γ, expert store $\mathcal{E}$

Initialize accumulator $\mathcal{A}$ and queue $\mathcal{Q}$
Generator and trainer run concurrently
Generator: launch an attempt in each idle rollout slot
On completion of (g_x, i), store $\langle \tau_i, R(\tau_i), \pi_i^{\text{gen}} \rangle$ in $\mathcal{A}[g_x][i]$; refill the slot
if $|\mathcal{A}[g_x]| = n$ **then**
    Boundary 1: complete-group source decision
    $\widetilde{G}_x \leftarrow \text{ordered\_pop}(\mathcal{A}[g_x])$; $z_x \leftarrow S_\gamma(\widetilde{G}_x)$
    Push $\text{PolicyRecord}(x, \widetilde{G}_x)$ if $z_x = \text{policy}$; otherwise push $\text{ExpertRecord}(x, \mathcal{E}[x])$ to $\mathcal{Q}$
end if
Boundary 2: training-input construction at learner entry
Dequeue source-fixed records and initialize B
for each record q **do**
    Evaluate its selected sequences at learner entry and add the corresponding input in Eq. 2 or 3 to B
end for
Trainer: $\theta \leftarrow \text{LearnerUpdate}(\theta, \mathcal{L}(B))$; publish θ

The expert input therefore contributes the gradient of a β_r-weighted negative log-likelihood.

These endpoint identities define the learning contract that the asynchronous handoff must preserve. Once the inputs are constructed, learner control no longer branches on source: both sources share one optimizer path and policy objective.

A policy-routed group contributes n sequences, whereas an expert-routed group contributes one. A source-specific denominator would therefore introduce another mixture rule after source selection. We instead apply the same fixed normalization to both sources and keep supervision strength explicit in β_r. The effective source mixture is not determined by routing frequency alone: row cardinality, token volume, and β_r also matter. Appendix A.3 quantifies this cascade under the shared normalization and distinguishes token share from gradient contribution. Appendix A.2 gives the remaining token-level details, including masking and stop-gradient construction.

Together, these identities and the shared normalization close the information dependency at input construction.

3.2 Complete-Group Decisions under Fully Asynchronous Execution

Section 3.1 established the learning contract that asynchronous execution must preserve. StreamWeave preserves it without allowing complete-group waiting to govern pipeline progress: the wait ends at the source decision, while a source-fixed record carries the required context through the queue to training-input construction.

Execute attempts independently and reconstruct each group locally. Each attempt executes independently and releases its slot upon completion; the generator immediately fills that slot rather than waiting for sibling attempts. The accumulator stores each result with its group identity and position and reconstructs the group in its original order once all n results arrive. A slow attempt therefore delays only its group's source decision: the complete-group requirement governs decision availability, not pipeline progress.

Make the complete-group source decision before the queue. Once the accumulator reconstructs a complete group, StreamWeave applies the source rule S_γ to select either the policy rollouts or the paired expert trajectory for training. The selected data, group outcomes, and per-rollout policy provenance enter the queue in one source-fixed record, so later consumption cannot revise the source decision.

Construct training inputs at learner entry. At learner entry, each dequeued source-fixed record is converted into the training input defined in Section 3.1. This placement keeps policy-dependent fields, including the learner-entry reference and rollout-to-entry correction, aligned with the learner-entry policy rather than freezing them before transport. The resulting input encodes the selected learning role and enters one policy objective without source-specific branching.

One asynchronous flow. The two queue boundaries confine composition-specific control while the generator produces independent attempts and the trainer optimizes prepared inputs through their existing interfaces. Together, they end complete-group waiting at the source decision and source-specific learner control at input construction, allowing the selected learning role to reach one policy-objective path without a pipeline-wide barrier. Appendix B follows source-fixed records through queueing and parameter refresh, showing how the handoff preserves the fixed source while retaining the existing asynchronous flow.

4 Experiments

Section 3 established how StreamWeave preserves the learning role selected by each complete group while confining the required wait to the source decision. We evaluate three questions: whether StreamWeave retains competitive mathematical and cross-domain reasoning, whether the costly all-failure region persists and expert supervision remains learning-relevant within it, and whether localizing complete-group waiting raises same-budget throughput under sustained load.

4.1 Experimental Setup

Training setting and control. StreamWeave is implemented in verl (Sheng et al. 2025), starts from Qwen2.5-Math-1.5B (Yang et al. 2024), and trains on prompts paired with verified expert trajectories from OpenR1-Math (Hugging Face 2025). For each prompt, the policy generates $n = 8$ rollouts with source threshold $\gamma = 0$, so the paired expert trajectory is used only when all eight rollouts fail. To isolate the role of expert supervision, we compare against Async RL (expert-off), which disables expert trajectories while retaining the same fully asynchronous execution stack. Appendices A.2–A.4 and C.1 specify the expert-signal scale, lag correction, normalization, and optimizer settings.

Type	Model	Async	Expert	AIME24	AIME25	AMC	MATH500	Minerva	Olympiad	Avg.
Basic	Base	–	–	6.6 ± 2.5	3.5 ± 1.6	31.2 ± 3.2	43.3 ± 1.4	10.9 ± 1.3	24.9 ± 1.2	20.1 ± 0.8
	Instruct	–	–	10.6 ± 4.6	9.4 ± 3.1	47.0 ± 4.4	75.5 ± 1.6	29.5 ± 2.4	40.4 ± 1.6	35.4 ± 1.3
Hybrid PT	HPT	–	✓	15.4 ± 5.7	12.6 ± 4.6	45.8 ± 4.5	78.0 ± 1.6	31.3 ± 2.4	43.2 ± 1.6	37.7 ± 1.5
	SRFT	–	✓	12.3 ± 4.2	10.4 ± 3.7	43.0 ± 4.0	71.6 ± 1.5	26.2 ± 2.1	38.5 ± 1.5	33.7 ± 1.3
	ReLIFT	–	✓	12.6 ± 4.9	8.1 ± 3.1	40.3 ± 4.3	74.6 ± 1.6	28.6 ± 2.3	39.5 ± 1.6	34.0 ± 1.3
	LUFFY	–	✓	15.1 ± 5.5	**14.0** ± 4.9	46.0 ± 4.5	77.5 ± 1.6	30.0 ± 2.4	**43.5** ± 1.6	37.7 ± 1.5
Fully Async	Async RL	✓	–	12.9 ± 5.3	7.9 ± 2.7	44.9 ± 4.4	75.8 ± 1.7	28.8 ± 2.4	39.5 ± 1.6	35.0 ± 1.4
Ours	**StreamWeave**	✓	✓	16.1 ± 5.5	13.0 ± 4.9	47.0 ± 4.5	**78.5** ± 1.5	33.0 ± 2.5	43.2 ± 1.6	**38.5** ± 1.5
Additional	Oat-Zero	–	–	**17.2** ± 5.6	12.6 ± 4.5	**49.6** ± 4.5	73.7 ± 1.7	30.1 ± 2.5	38.2 ± 1.6	36.9 ± 1.5
	AceMath	–	✓	11.3 ± 4.5	6.6 ± 2.9	45.4 ± 4.5	78.1 ± 1.6	**34.8** ± 2.6	39.4 ± 1.6	35.9 ± 1.3
	ExGRPO	–	–	12.1 ± 4.1	9.0 ± 3.3	46.3 ± 4.4	73.2 ± 1.7	27.2 ± 2.3	37.3 ± 1.6	34.2 ± 1.3
	EMPO	–	–	14.1 ± 4.6	8.0 ± 2.7	45.5 ± 4.3	72.5 ± 1.7	25.1 ± 2.2	35.8 ± 1.6	33.5 ± 1.3
	DFT	–	✓	6.0 ± 3.9	4.3 ± 2.5	32.0 ± 4.3	63.5 ± 2.0	26.1 ± 2.3	29.0 ± 1.5	26.8 ± 1.2

Table 1: Fixed-checkpoint mathematical reasoning. Async and Expert denote fully asynchronous training and external expert trajectories; AIME24, AIME25, and AMC use mean@32, the others mean@8. Avg. is the unweighted benchmark mean; gray ± terms are problem-level SE, propagated for Avg. Bold and underline mark the highest and second-highest point estimates.

Evaluation. We evaluate fixed checkpoints on six mathematical reasoning benchmarks (Hendrycks et al. 2021; Lightman et al. 2024; Lewkowycz et al. 2022; He et al. 2024) and three cross-domain reasoning benchmarks (Clark et al. 2018; Rein et al. 2024; Wang et al. 2024) (Tables 1 and 3). We evaluate every reported checkpoint with the same sampling configuration, Math-Verify grader (Kydlíček 2025), and rounding procedure; Appendix C.2 records benchmark-specific sample counts and checkpoint provenance for this fixed-checkpoint protocol rather than selected-peak reporting. Comparisons include the base and instruct checkpoints, HPT as the synchronous same-rule quality reference, and official author-released checkpoints for SRFT, ReLIFT, Oat-Zero, LUFFY, ExGRPO, EMPO, AceMath, and DFT (Liu et al. 2025a; Yan et al. 2025; Fu et al. 2026; Ma et al. 2026; Zhan et al. 2026; Zhang et al. 2025; Liu et al. 2025b; Wu et al. 2026).

4.2 Learning Effectiveness

Overall mathematical reasoning performance. Table 1 shows that StreamWeave reaches a six-benchmark average of 38.5, compared with 35.0 for the same-stack expert-off control and 37.7 for HPT, the synchronous same-rule quality reference. Its 38.5 point estimate is also the highest among the evaluated checkpoints. A decoding-cap sweep shows that the expert-off control is not constrained by the cap: quadrupling it changes the six-benchmark average by only 0.02 points, and no sample reaches the largest limit. StreamWeave reaches 38.5 with a 4,096-token cap; under the common 8,192-token protocol, it uses 48% of LUFFY's mean response tokens at a comparable score (Appendix C.2).

A costly hard region. Under the $0/8$ rule, an all-failure group selects the expert source because its policy rollouts contain no group-relative reward contrast. To test whether this region also carries greater generation workload, Table 2 partitions training groups by outcome.

All-failure groups consume a larger share of generated response tokens than their share of groups and exhibit longer rollouts and a wider within-group completion spread. Missing reward contrast, generation workload, and completion-tail pressure therefore concentrate in the same groups: those needing expert supervision also generate the long, uneven rollouts that benefit most from asynchronous execution.

Metric	Any success	**All failure**
Group frequency	77.6%	**22.4%**
Generated-token share	73.2%	**26.8%**
Mean rollout length (tokens)	1,347	**1,713**
Completion spread (s)	7.7	**11.0**

Table 2: Pre-selection generator census; completion spread is the within-group slowest-minus-fastest request time.

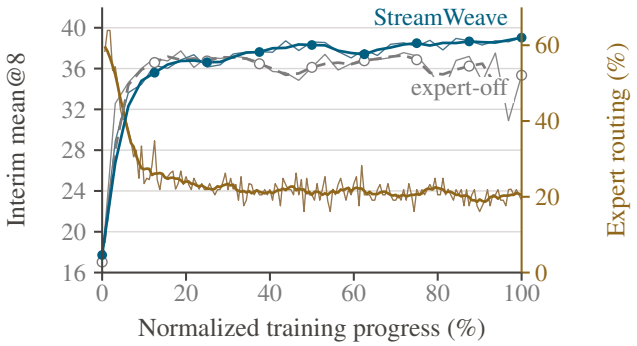

Figure 2: Learning dynamics; thick curves are smoothed trends over normalized training progress.

Expert routing falls rapidly as the policy improves but settles into a persistent tail of roughly 20%, leaving a residual region without reward contrast late in training (Figure 2). Throughout this tail, selected expert inputs enter the one primary update defined in Section 3.1.

StreamWeave and the expert-off control share the execution stack, policy objective, optimizer, prompt population, and expert access; at an all-failure encounter, they differ only in whether the selected expert input is consumed. The two runs improve similarly at first and then separate: on a held-

Model	ARC-C	GPQA-D	MMLU-Pro	Avg.
Base	5.7 ± 0.3	3.6 ± 0.4	3.3 ± 0.2	4.2 ± 0.2
Instruct	54.6 ± 1.1	27.7 ± 1.8	30.3 ± 0.4	37.5 ± 0.7
HPT	$\underline{60.5} \pm 1.0$	28.8 ± 1.6	33.9 ± 0.4	$\underline{41.1} \pm 0.6$
SRFT	48.0 ± 0.9	24.2 ± 1.5	28.0 ± 0.4	33.4 ± 0.6
ReLIFT	52.8 ± 0.9	24.5 ± 1.4	30.2 ± 0.4	35.8 ± 0.6
LUFFY	56.5 ± 1.0	27.0 ± 1.5	$\underline{34.5} \pm 0.4$	39.3 ± 0.6
Async RL	$\mathbf{60.8} \pm 1.0$	$\mathbf{30.6} \pm 1.9$	31.7 ± 0.4	$\underline{41.1} \pm 0.7$
StreamWeave	60.2 ± 1.0	$\underline{30.3} \pm 1.6$	33.2 ± 0.4	$\mathbf{41.2} \pm 0.6$
Oat-Zero	41.9 ± 1.0	13.5 ± 1.4	21.5 ± 0.4	25.6 ± 0.6
AceMath	59.9 ± 1.1	25.9 ± 1.7	$\mathbf{34.7} \pm 0.4$	40.2 ± 0.7
ExGRPO	58.7 ± 1.0	28.3 ± 1.7	29.2 ± 0.4	38.7 ± 0.7
EMPO	24.8 ± 0.8	12.3 ± 1.2	16.2 ± 0.3	17.8 ± 0.5
DFT	44.8 ± 1.2	22.0 ± 1.8	24.0 ± 0.4	30.2 ± 0.7

Table 3: Fixed-checkpoint cross-domain reasoning. Avg. is the unweighted benchmark mean; gray $\pm$ terms are problem-level SE, propagated for Avg.

out panel the all-failure rate ends at 32.4% for StreamWeave against 37.0% for the control, and StreamWeave also ends higher on the same panel's average score.

To separate a local effect on directly routed prompts from a policy-level one, we match prompts by their number of successes at first encounter and compare later encounters. Weighting each initial-k stratum by its number of matched prompts, StreamWeave's improvement from the first encounter to the second exceeds the expert-off control by $+0.169$ successful rollouts per eight-attempt group. The difference is positive in all nine strata $k = 0, \ldots, 8$, is largest in the partial-success region, and remains $+0.159$ at the third encounter. The pattern is consistent with selective expert contributions supporting both the paired prompt and broader acquisition and retention in the shared policy. Appendix C.3 reports the matched and held-out populations, the full stratum-level estimates with standard errors, and the follow-up encounters. At matched prompt-group consumption, StreamWeave retains a 4.8-percentage-point lower late all-failure rate; the comparison remains limited to two fixed training runs rather than a randomized or multi-seed study.

Cross-domain reasoning. To test whether math-focused expert supervision narrows broader reasoning, we evaluate the same checkpoints on ARC-Challenge, GPQA-Diamond, and MMLU-Pro (Table 3).

StreamWeave remains at the same aggregate cross-domain level as Async RL (expert-off) and HPT. The gain over the expert-off control is therefore concentrated in mathematics, with no clear cross-domain reduction.

Choosing the policy-update rule. Decoupled PPO separates PPO Clip-Higher at learner entry from rollout-to-entry correction, allowing us to test whether clipping removes useful policy-rollout gradients when expert inputs update the same policy (Section 3.1). Holding expert routing, policy-lag correction, and asynchronous execution fixed, we replaced only PPO clipping with a CISPO-style rule that retains gradients outside the PPO clipping range (MiniMax 2025). The variant averages 36.8 on the six mathematical reasoning benchmarks, below 38.5 for StreamWeave, and making the queue fresher reduces early policy lag without removing the variant's later instability. Thus, correcting policy lag does not make PPO clipping redundant in this setting; Appendix D reports the full ablation and training-history evidence.

Metric	Ref.	**Ours**	Change
Generation workload (expert-off reference)			
Response length (tokens)	896	**1,429**	$1.59\times$
Longest rollout/group (s)	6.59	**12.79**	$1.94\times$
Adjacent-version groups	2.03%	**7.00%**	$3.46\times$
End-to-end execution (synchronous reference)			
128-group time (s) $\downarrow$	46.0	**28.0**	-39%
Throughput (groups/s) $\uparrow$	2.78	**4.58**	$1.64\times$

Table 4: Generation workload and execution; throughput is pre-routing prompt groups per training-loop second, excluding validation and checkpointing.

4.3 Execution Efficiency with Expert Supervision

Because expert supervision remains active late into training (Section 4.2), the generation workload and completion-tail pressure observed in this regime are persistent execution conditions rather than a transient startup effect. Two questions follow: whether localizing the wait avoids the execution cost implied by complete groups, and whether it improves end-to-end throughput at equal GPU budget.

On the same asynchronous stack, the run with expert supervision exhibits longer responses, wider group tails, and a higher incidence of groups containing rollouts from adjacent policy versions than the expert-off control (Table 4).

This temporal exposure is not uniform across group outcomes. Within StreamWeave, 10.0% of the all-failure groups ultimately assigned to the expert source contain rollouts from adjacent policy versions, the highest rate among the routes. The region that requires expert supervision therefore interacts most strongly with asynchronous execution not only through its absent reward contrast, but also through when its complete group becomes available.

When all rollouts in a long and uneven group share one execution phase, the slowest trajectory governs pipeline progress. In the synchronous trace, requests complete in 7.6 seconds on average, but the slowest lasts 23.7 seconds and the generation phase, including post-processing, ends at 25.1 seconds. Earlier completions cannot launch the next attempt before the phase closes, and model training starts only after generation. The completion tail therefore constrains both subsequent rollout work and training.

To quantify group-level pinning, we consider a counterfactual that holds only a group's eight execution slots until its slowest attempt completes and attributes no other pipeline stall to pinning. From the observed attempt durations, this schedule would consume $1.80\times$ as many rollout slot-seconds as StreamWeave's executed schedule, and $1.53\times$ as many in the expert-off control (Table 5). Reconstructed from the same trace, the observed and pinned schedules also imply different concurrent-attempt demands. Executed concurrency

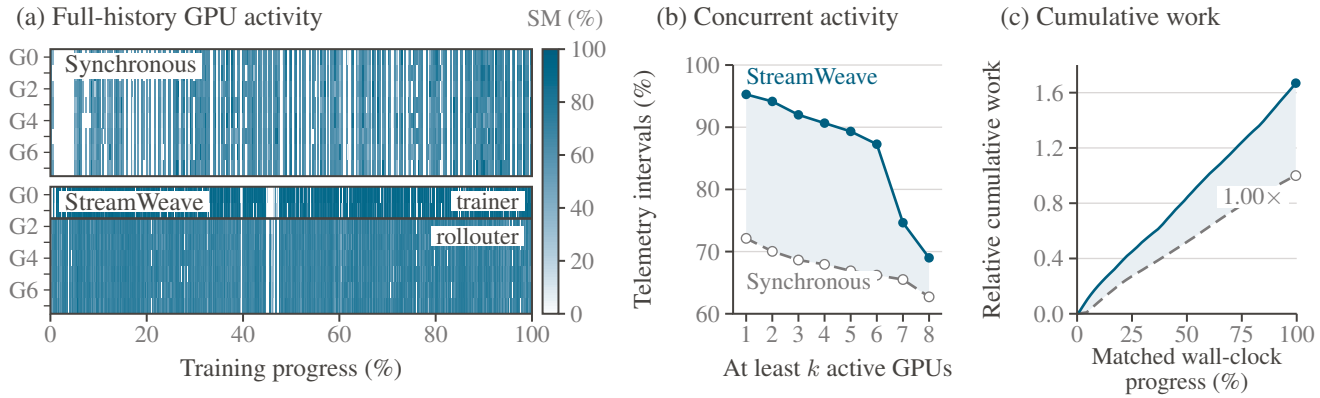

Figure 3: Same-budget execution. Panels (a)–(b) normalize each run's non-validation history; panel (c) compares cumulative work over a matched wall-clock interval, normalized to the synchronous endpoint.

	If slots were pinned	**As executed**
Slot-seconds per group	$8 \max_i d_i$	$\sum_i d_i$
Relative slot-seconds required	1.80×	1.00×
Concurrent attempts, Q1 → Q4	521 → 850	**363 → 372**
Concurrent attempts, p95	983	**442**

Table 5: Slot-pinning counterfactual from observed durations: d_i is attempt duration, quartiles order parameter versions by pinning cost, and runtime capacity is 768 attempts.

stays below the runtime's 768-attempt limit at every quantile we checked, peaking at 507, whereas pinning demand reaches 983 at its 95th percentile. The same fixed rollout pool can therefore sustain the observed admission rate, but not the pinned schedule at that rate.

What the complete-group requirement consumes directly is completed-data storage rather than active execution slots. We normalize both occupancies by 3,072 completed attempts, equivalent to the 384-group completed queue at eight attempts per group. On a time-weighted average, completed attempts held inside unfinished groups occupy 7.2% of this capacity, compared with 69.3% for complete groups awaiting the trainer. Consistently, the rollout producer is held off from submitting new work for 23.2% of the parameter-version interval, while the trainer's blocking wait accounts for 2.1% of the same interval; the dominant pressure follows trainer consumption rather than group reconstruction.

Figure 3 shows the hardware-level signature of this design: intervals in which no GPU exceeds the activity threshold fall from 27.9% to 4.7%, the mean number of concurrently active GPUs rises from 5.40 to 6.92, and more prompt-group work accumulates over the same wall-clock interval.

StreamWeave reduces the processing time for 128 prompt groups from 46 to 28 seconds and achieves 1.64× prompt-group throughput under the same GPU budget. Normalized by the response tokens the trainer consumes, the most conservative definition we checked, the gain is 1.30×; expert routing lets StreamWeave consume 21% fewer response tokens per prompt group. Synchronous execution spends 25.1 of its 46.0 seconds generating with all eight GPUs and the remaining 20.9 on computation that cannot begin until generation closes, whereas StreamWeave completes both within 28.0 seconds with six GPUs on its rollouter. The measured gain comes from generation–training overlap and immediate slot refill; Section 3.2 establishes how StreamWeave preserves complete-group source decisions without obstructing either mechanism. The pinning counterfactual and occupancy analysis validate this bounded scope in execution: pinning would consume additional rollout capacity, whereas incomplete groups consume completed-data storage rather than active rollout slots.

Appendix C.4 gives the per-version comparison behind Table 5, the synchronous request timing, and the route-level version span, treating the version-level statistics as descriptive rather than inferential. Appendix C.5 records the throughput protocol and shows the gain staying above 1.15× across every work unit and time window we checked.

5 Conclusion

StreamWeave confines complete-group waiting to the source decision and preserves source-specific context only until training-input construction. These boundaries preserve the selected learning role under one policy objective without making group completion a pipeline-wide synchronization barrier. Experiments show that the motivating conditions persist: generation workload and absent reward contrast concentrate in all-failure groups, expert demand remains late into training, and the observed learning difference extends beyond directly routed prompts. Under these conditions, StreamWeave matches the synchronous quality reference, retains competitive cross-domain reasoning, and delivers 1.64× prompt-group throughput under the same GPU budget. Our direct empirical scope is one backbone, one training seed, an 8-GPU budget, and math-centered two-source RLVR with a group-outcome-based source rule. StreamWeave preserves complete-group source decisions and learning roles without placing group completion on the critical path.

References

Ball, P. J.; Smith, L.; Kostrikov, I.; and Levine, S. 2023. Efficient Online Reinforcement Learning with Offline Data. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, 1577–1594.

Clark, P.; Cowhey, I.; Etzioni, O.; Khot, T.; Sabharwal, A.; Schoenick, C.; and Tafjord, O. 2018. Think You Have Solved Question Answering? Try ARC, the AI2 Reasoning Challenge. arXiv:1803.05457.

Fu, W.; Gao, J.; Shen, X.; Zhu, C.; Mei, Z.; He, C.; Xu, S.; Wei, G.; Mei, J.; Wang, J.; Yang, T.; Yuan, B.; and Wu, Y. 2025. AReaL: A Large-Scale Asynchronous Reinforcement Learning System for Language Reasoning. In *Advances in Neural Information Processing Systems*, volume 38, 36256–36282. Curran Associates, Inc.

Fu, Y.; Chen, T.; Chai, J.; Wang, X.; Tu, S.; Yin, G.; Lin, W.; Zhang, Q.; Zhu, Y.; and Zhao, D. 2026. SRFT: A Single-Stage Method with Supervised and Reinforcement Fine-Tuning for Reasoning. In *International Conference on Learning Representations*.

He, C.; Luo, R.; Bai, Y.; Hu, S.; Thai, Z. L.; Shen, J.; Hu, J.; Han, X.; Huang, Y.; Zhang, Y.; Liu, J.; Qi, L.; Liu, Z.; and Sun, M. 2024. OlympiadBench: A Challenging Benchmark for Promoting AGI with Olympiad-Level Bilingual Multimodal Scientific Problems. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 3828–3850. Association for Computational Linguistics.

Hendrycks, D.; Burns, C.; Kadavath, S.; Arora, A.; Basart, S.; Tang, E.; Song, D.; and Steinhardt, J. 2021. Measuring Mathematical Problem Solving with the MATH Dataset. In *Advances in Neural Information Processing Systems Track on Datasets and Benchmarks*.

Hester, T.; Vecerik, M.; Pietquin, O.; Lanctot, M.; Schaul, T.; Piot, B.; Horgan, D.; Quan, J.; Sendonaris, A.; Osband, I.; Dulac-Arnold, G.; Agapiou, J.; Leibo, J. Z.; and Gruslys, A. 2018. Deep Q-learning From Demonstrations. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 32, 3223–3230.

Hugging Face. 2025. Open R1: A fully open reproduction of DeepSeek-R1. https://github.com/huggingface/open-r1. OpenR1-Math-220k dataset: https://huggingface.co/datasets/open-r1/OpenR1-Math-220k.

Kydlíček, H. 2025. Math-Verify: Math Verification Library. https://github.com/huggingface/Math-Verify.

Lewkowycz, A.; Andreassen, A.; Dohan, D.; Dyer, E.; Michalewski, H.; Ramasesh, V.; Slone, A.; Anil, C.; Schlag, I.; Gutman-Solo, T.; Wu, Y.; Neyshabur, B.; Gur-Ari, G.; and Misra, V. 2022. Solving Quantitative Reasoning Problems with Language Models. In *Advances in Neural Information Processing Systems*, volume 35.

Lightman, H.; Kosaraju, V.; Burda, Y.; Edwards, H.; Baker, B.; Lee, T.; Leike, J.; Schulman, J.; Sutskever, I.; and Cobbe, K. 2024. Let's Verify Step by Step. In *International Conference on Learning Representations*.

Liu, Z.; Chen, C.; Li, W.; Qi, P.; Pang, T.; Du, C.; Lee, W. S.; and Lin, M. 2025a. Understanding R1-Zero-Like Training: A Critical Perspective. In *Second Conference on Language Modeling*.

Liu, Z.; Chen, Y.; Shoeybi, M.; Catanzaro, B.; and Ping, W. 2025b. AceMath: Advancing Frontier Math Reasoning with Post-Training and Reward Modeling. In *Findings of the Association for Computational Linguistics: ACL 2025*, 3993–4015. Vienna, Austria: Association for Computational Linguistics.

Lv, X.; Zuo, Y.; Sun, Y.; Liu, H.; Wei, Y.; Chen, Z.; He, L.; Zhu, X.; Zhang, K.; Wang, B.; Ding, N.; and Zhou, B. 2025. Towards a Unified View of Large Language Model Post-Training. arXiv:2509.04419.

Ma, L.; Liang, H.; Qiang, M.; Tang, L.; Ma, X.; Wong, Z. H.; Niu, J.; Shen, C.; He, R.; Li, Y.; Cui, B.; and Zhang, W. 2026. Learning What Reinforcement Learning Can't: Interleaved Online Fine-Tuning for Hardest Questions. In *International Conference on Learning Representations*.

MiniMax. 2025. MiniMax-M1: Scaling Test-Time Compute Efficiently with Lightning Attention. arXiv:2506.13585.

Noukhovitch, M.; Huang, S.; Xhonneux, S.; Hosseini, A.; Agarwal, R.; and Courville, A. 2025. Asynchronous RLHF: Faster and More Efficient Off-Policy RL for Language Models. In *International Conference on Learning Representations*.

Rein, D.; Hou, B. L.; Stickland, A. C.; Petty, J.; Pang, R. Y.; Dirani, J.; Michael, J.; and Bowman, S. R. 2024. GPQA: A Graduate-Level Google-Proof Q&A Benchmark. In *First Conference on Language Modeling*.

Schulman, J.; Wolski, F.; Dhariwal, P.; Radford, A.; and Klimov, O. 2017. Proximal Policy Optimization Algorithms. arXiv:1707.06347.

Shao, Z.; Wang, P.; Zhu, Q.; Xu, R.; Song, J.; Bi, X.; Zhang, H.; Zhang, M.; Li, Y. K.; Wu, Y.; and Guo, D. 2024. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. arXiv:2402.03300.

Sheng, G.; Tong, Y.; Wan, B.; Zhang, W.; Jia, C.; Wu, X.; Wu, Y.; Li, X.; Zhang, C.; Peng, Y.; Lin, H.; Liu, X.; and Wu, C. 2026. Laminar: A Scalable Asynchronous RL Post-Training Framework. In *Proceedings of the 21st European Conference on Computer Systems*, EuroSys '26, 400–422. New York, NY, USA: Association for Computing Machinery.

Sheng, G.; Zhang, C.; Ye, Z.; Wu, X.; Zhang, W.; Zhang, R.; Peng, Y.; Lin, H.; and Wu, C. 2025. HybridFlow: A Flexible and Efficient RLHF Framework. In *Proceedings of the Twentieth European Conference on Computer Systems*, EuroSys '25, 1279–1297. New York, NY, USA: Association for Computing Machinery.

Wang, Y.; Ma, X.; Zhang, G.; Ni, Y.; Chandra, A.; Guo, S.; Ren, W.; Arulraj, A.; He, X.; Jiang, Z.; Li, T.; Ku, M.; Wang, K.; Zhuang, A.; Fan, R.; Yue, X.; and Chen, W. 2024. MMLU-Pro: A More Robust and Challenging Multi-Task Language Understanding Benchmark. In *Advances in Neural Information Processing Systems Track on Datasets and Benchmarks*.

Wu, Y.; Zhou, Y.; Zhou, Z.; Peng, Y.; Ye, X.; Hu, X.; Zhu, W.; Qi, L.; Yang, M.-H.; and Yang, X. 2026. On the Generalization of SFT: A Reinforcement Learning Perspective with Reward Rectification. In *International Conference on Learning Representations*.

Yan, J.; Li, Y.; Hu, Z.; Wang, Z.; Cui, G.; Qu, X.; Cheng, Y.; and Zhang, Y. 2025. Learning to Reason under Off-Policy Guidance. In *Advances in Neural Information Processing Systems*.

Yang, A.; Zhang, B.; Hui, B.; Gao, B.; Yu, B.; Li, C.; Liu, D.; Tu, J.; Zhou, J.; Lin, J.; Lu, K.; Xue, M.; Lin, R.; Liu, T.; Ren, X.; and Zhang, Z. 2024. Qwen2.5-Math Technical Report: Toward Mathematical Expert Model via Self-Improvement. arXiv:2409.12122.

Zhan, R.; Li, Y.; Wang, Z.; Qu, X.; Liu, D.; Shao, J.; Wong, D. F.; and Cheng, Y. 2026. ExGRPO: Learning to Reason from Experience. In *International Conference on Learning Representations*.

Zhang, Q.; Wu, H.; Zhang, C.; Zhao, P.; and Bian, Y. 2025. Right Question is Already Half the Answer: Fully Unsupervised LLM Reasoning Incentivization. In *Advances in Neural Information Processing Systems*.

Zhang, W.; Xie, Y.; Sun, Y.; Chen, Y.; Wang, G.; Li, Y.; Ding, B.; and Zhou, J. 2026. On-Policy RL Meets Off-Policy Experts: Harmonizing Supervised Fine-Tuning and Reinforcement Learning via Dynamic Weighting. In *International Conference on Learning Representations*.